

Documentación Técnica: Sistema Backend Bancat

1. Arquitectura General y Flujo de Procesamiento

El ecosistema de Bancat se fundamenta en una arquitectura moderna basada en **FastAPI** y **Uvicorn**, una combinación diseñada para maximizar el rendimiento a través del manejo eficiente de operaciones asíncronas bajo el estándar ASGI. En un entorno bancario de alta concurrencia, esta infraestructura permite procesar múltiples peticiones de evaluación de riesgo sin bloquear el hilo principal, asegurando una latencia mínima y una alta disponibilidad.

El flujo estándar de una petición dentro del sistema, específicamente para la creación de solicitudes mediante `POST /api/solicitar-credito`, sigue una secuencia lógica de validación y persistencia:

1. **Recepción y Parsing:** Uvicorn recibe la petición HTTP y la transfiere a FastAPI, que actúa como el orquestador del ciclo de vida del request.
2. **Validación de Esquema:** El cuerpo de la petición (*body*) se valida inmediatamente contra el modelo `Pydantic ClienteNuevo`, garantizando que solo datos bien estructurados avancen en el pipeline.
3. **Inyección de Dependencias:** Se utiliza el patrón `Depends(get_db)` para inyectar una sesión de SQLAlchemy gestionada, asegurando que cada hilo de ejecución tenga su propia conexión aislada.
4. **Lógica de Negocio y Scoring:** La función `crear_cliente()` verifica la unicidad del correo, ejecuta el motor de Machine Learning para el scoring crediticio y prepara la persistencia.
5. **Serialización de Salida:** La respuesta se filtra y serializa mediante el `response_model` de `ClienteResponse`, enviando al cliente únicamente los campos autorizados.

La implementación de `Depends(get_db)` y el uso de modelos Pydantic no son meras convenciones; son garantías de **integridad de datos y seguridad**. Actúan como un firewall de inyección de dependencias que previene fugas de conexiones y asegura que la capa de persistencia nunca reciba datos que no cumplan con el contrato estricto de la API.

2. Infraestructura del Servidor: FastAPI y Uvicorn

La elección estratégica de FastAPI responde a la necesidad de contar con un sistema autodocumentado y altamente escalable. La generación automática de especificaciones **OpenAPI/Swagger** en el endpoint `/docs` facilita la integración continua y la auditoría técnica de los servicios.

Gestión de Estado y Servido de Archivos

- **Gestión de Sesiones:** El sistema emplea un patrón generador con la cláusula `finally` para el manejo de sesiones de base de datos. Esto asegura que toda sesión abierta se cierre rigurosamente al concluir la petición, eliminando riesgos de agotamiento de pool de conexiones.
- **SPA Fallback para SvelteKit:** El servidor está configurado para priorizar las rutas de la API (`/api/*`). Para las rutas de interfaz, implementa un *catch-all* que sirve el build estático de SvelteKit, integrando protecciones contra ataques de **path traversal** para evitar el acceso a archivos fuera del directorio de despliegue permitido.

La validación automática de Pydantic reduce drásticamente los errores en tiempo de ejecución al actuar como una capa de validación previa a la lógica de negocio. Esto no solo mejora la estabilidad del sistema, sino que eleva la **Experiencia del Desarrollador (DX)** al proporcionar mensajes de error claros y estructurados, permitiendo que la gestión del servidor se enfoque en la orquestación eficiente del estado y la persistencia.

3. Capa de Persistencia: SQLite y SQLAlchemy

La persistencia de Bancat se apoya en un motor **SQLite** que gestiona el archivo físico `bancat.db`, garantizando la durabilidad de los datos ante reinicios del servicio. SQLAlchemy actúa como el ORM encargado de abstraer la complejidad del SQL tradicional hacia modelos de objetos eficientes.

Modelado de Datos: ClienteDB

El modelo central almacena tanto el perfil socioeconómico como los resultados derivados del motor de scoring para optimizar las consultas de lectura.

Campo	Tipo	Notas / Lógica
id	Integer (PK)	Autoincremental; sujeto a reindexado.
nombre, correo	String	Correo electrónico único e indexado para búsquedas rápidas.
edad, ingresos, etc.	Mixto	Atributos (features) utilizados por el modelo de ML.
producto_solicitado	String	Producto de interés inicial (nullable).
archivado	Boolean	Implementación de soft-delete para historial y auditoría.
probabilidad_riesgo	Float	Output directo (PD) del modelo de Machine Learning.
nivel_riesgo	String	Categorización semántica del riesgo (Bajo a Muy Alto).
monto_maximo	Float	Capacidad de crédito total calculada por el motor.
*_json	Text	Almacena productos, evaluaciones y valores SHAP serializados.

Además, el sistema incluye un **derivado en lectura**: la `capacidad_pago_mensual`, calculada como el 30% de los ingresos menos las cuotas de deudas existentes, asegurando que el perfil financiero sea transparente para el frontend. El modelo mantiene una relación 1-N con **ProductoBancarioDB**, permitiendo la eliminación en cascada si se requiere una depuración total del registro.

Lógica de Reindexado y Higiene

Para mantener la coherencia referencial y la higiene del sistema tras borrados definitivos, se ejecuta una estrategia de reindexado en tres pasos:

1. **Desplazamiento (OFFSET)**: Se mueven todos los IDs a un rango superior para evitar colisiones durante la reasignación.

2. **Reasignación Secuencial:** Se actualizan los IDs de 1 a N, propagando el cambio a las llaves foráneas de productos.
3. **Invalidación de Cache:** Se invoca `db.expire_all()` para limpiar la *identity map* de SQLAlchemy, forzando al ORM a reconocer los nuevos identificadores y evitar errores de inconsistencia en tiempo de ejecución.

Esta combinación de *soft-delete* y reindexado garantiza un sistema compacto y auditable, donde la integridad referencial es absoluta.

4. Configuración y Constantes del Sistema

La centralización de variables operativas permite ajustar el modelo de negocio y los parámetros de seguridad sin modificar el núcleo del software.

Variable	Valor	Propósito Operativo
CORREO_REGEX	Regex Estándar	Validación de formato de correos electrónicos.
MAX_DEBT_TO_INCOME_RATIO	60	Límite máximo de endeudamiento permitido (%).
CAPACIDAD_PAGO_RATIO	0.30	Factor de ingreso disponible para cuotas (30%).
SESSION_SECRET	String (Firma)	Clave para el cifrado de cookies de sesión.
ADMIN_USUARIO	admin	Usuario maestro para acceso administrativo.
ADMIN_PASSWORD	admin123	Credencial demo para validación de acceso.
SESSION_MAX_AGE	7 días	Tiempo de vida de la sesión administrativa.
TASAS_PRODUCTO	Diccionario	Configuración de Risk-Based Pricing por producto.

El uso de **Risk-Based Pricing** a través de `TASAS_PRODUCTO` optimiza la rentabilidad al asignar tasas de interés proporcionales al perfil de riesgo detectado, vinculando la configuración directamente con la lógica de validación de los esquemas de datos.

5. Modelado de Datos y Validación (Schemas Pydantic)

La separación entre la validación de entrada (Payload) y la serialización de salida (Response) es fundamental para la seguridad del contrato de la API.

- **Validación de Entrada (ClientePayload):** Implementa restricciones de rango mediante `Field` (ej. edad entre 18 y 80 años, ingresos mayores a cero).
- **Uniones Discriminadas:** Para los productos bancarios, Pydantic utiliza una *discriminated union*, aplicando validaciones específicas (como verificar que el saldo utilizado no exceda el cupo) según el tipo de producto.
- **Seguridad y Filtrado:** El `response_model` actúa como un filtro de seguridad en los endpoints, garantizando que campos sensibles o de control interno no sean expuestos al frontend.

Los validadores cruzados (`@model_validator`) aseguran que reglas complejas, como la relación entre deudas totales e ingresos, se cumplan estrictamente antes de que los datos toquen la base de datos, manteniendo la integridad del sistema.

6. Operaciones CRUD y Lógica de Negocio

Los endpoints materializan las reglas financieras de Bancat, gestionando el ciclo de vida del cliente.

Método	Ruta	Operación
POST	/api/clientes	Creación y ejecución de scoring crediticio.
GET	/api/clientes/estadisticas	Cálculo de agregados en tiempo real (debe ir antes que ID).
GET	/api/clientes/{id}	Recuperación de perfil por identificador único.
PUT	/api/clientes/{id}	Actualización de perfil y recálculo total de scoring.
DELETE	/api/clientes/{id}	Borrado en dos fases: Archivando o Eliminación definitiva.

La precedencia de la ruta de estadísticas sobre la de IDs es crítica para evitar conflictos de parseo en FastAPI. Asimismo, el recálculo de estadísticas en cada petición garantiza que el dashboard empresarial refleje siempre la realidad financiera más reciente del sistema.

7. Validación de Negocio y Manejo de Excepciones

El sistema utiliza excepciones semánticas para comunicar fallos, permitiendo una gestión de errores resiliente en el lado del cliente.

Código HTTP	Situación Específica en Bancat
400 / 422	Error de Pydantic (datos inválidos) o violación de reglas de negocio.
401	Error en las credenciales de administración (Login fallido).
404	Recurso (cliente o métricas) no encontrado en el sistema.
409	Conflicto por duplicidad de correo o número de producto bancario.

El mapeo detallado de estos códigos permite que el frontend, a través de su componente `ApiError`, proporcione retroalimentación accionable al usuario. Las validaciones de Pydantic que fallan lanzan automáticamente un **422 Unprocessable Entity**, detallando exactamente qué campo no cumple con los requisitos.

8. Inteligencia Artificial: Modelo de Machine Learning y Scoring

El motor de evaluación crediticia es el cerebro detrás de la viabilidad financiera de cada solicitud en Bancat.

Características del Modelo (Features)

El modelo de inferencia procesa **10 features** clave para determinar el perfil de riesgo:

- Edad y nivel de ingresos.
- Estado civil y tipo de empleo.
- Deudas existentes y relación deuda/ingreso.
- Saldo en cuentas y patrimonio total.
- Historial de pagos y tiempo de empleo actual.

Entrenamiento y Rendimiento

Entrenado con **6,000 muestras** calibradas para el mercado colombiano, el modelo utiliza un etiquetado probabilístico de Probabilidad de Incumplimiento (PD), evitando etiquetas determinísticas simplistas.

Métrica	Valor
AUC-ROC	0.823
Accuracy	80.9%
F1-score	0.490

Un AUC de 0.823 es indicativo de un modelo altamente robusto en el sector financiero. Este valor refleja una capacidad de discriminación realista entre clientes solventes y riesgosos, evitando el sobreajuste común en modelos con etiquetas artificiales.

Explicabilidad y Resiliencia

Para garantizar la transparencia, el sistema integra SHAP (DeepExplainer), que cuantifica la contribución de cada feature a la decisión final. El sistema implementa un mecanismo de degradación segura: si el módulo SHAP no puede inicializarse o falla, el sistema continúa procesando el scoring crediticio sin interrumpir la operación, omitiendo únicamente la visualización de explicabilidad detallada.

Conclusión

La arquitectura de Bancat representa una solución integral y robusta, donde la precisión del Machine Learning se une a una infraestructura backend segura y escalable, garantizando decisiones financieras confiables en tiempo real.